-

Form4
Capstone Design Implementation
Title :

GROUP MEMBER:

No.	Student Name	Student ID
1.	…. (Leader)	
2.	…. (Member 1)	
3.	…. (Member 2)	

Advisor : …….

Submitted for
Capstone Design Project
to Faculty of Computer Science
President University

TABLE OF CONTENT

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

Name : 1. Student Name – NIM

2. Student Name – NIM

3. Student Name – NIM

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled "**TITLE**" is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, January 2024

Signer 1	Signer 2	Signer 3
Student Name – NIM	Student Name – NIM	Student Name – NIM

SCREENSHOT OF ZEROGPT

PART 4
IMPLEMENTATION

Consists of:

A. **DESIGNS IMPLEMENTATION**
B. **PRODUCT DISPLAY**
C. **COMPONENT COST ANALYSIS**
D. **FUNCTIONAL TESTING**
E. **MANUAL GUIDE**

A. DESIGNS IMPLEMENTATION

1. Functions/Procedure/Class implementation based on the design that has been explained in Form 3, Part B. Hierarchical/Iterative Design. Explain the code for each part

2. Database implementation based on the design that has been explained in Form 3, Part B. Hierarchical/Iterative Design

3. User Interface implementation based on the design that has been explained in Form 3, Part B. Hierarchical/Iterative Design

4. Hardware implementation (if any).

5. Integration among every module based on the implementation explained in Parts 1, 2, 3, and 4 above.

EXAMPLE ONLY!

1. Functions/Procedure/Class Implementation

1. Post System

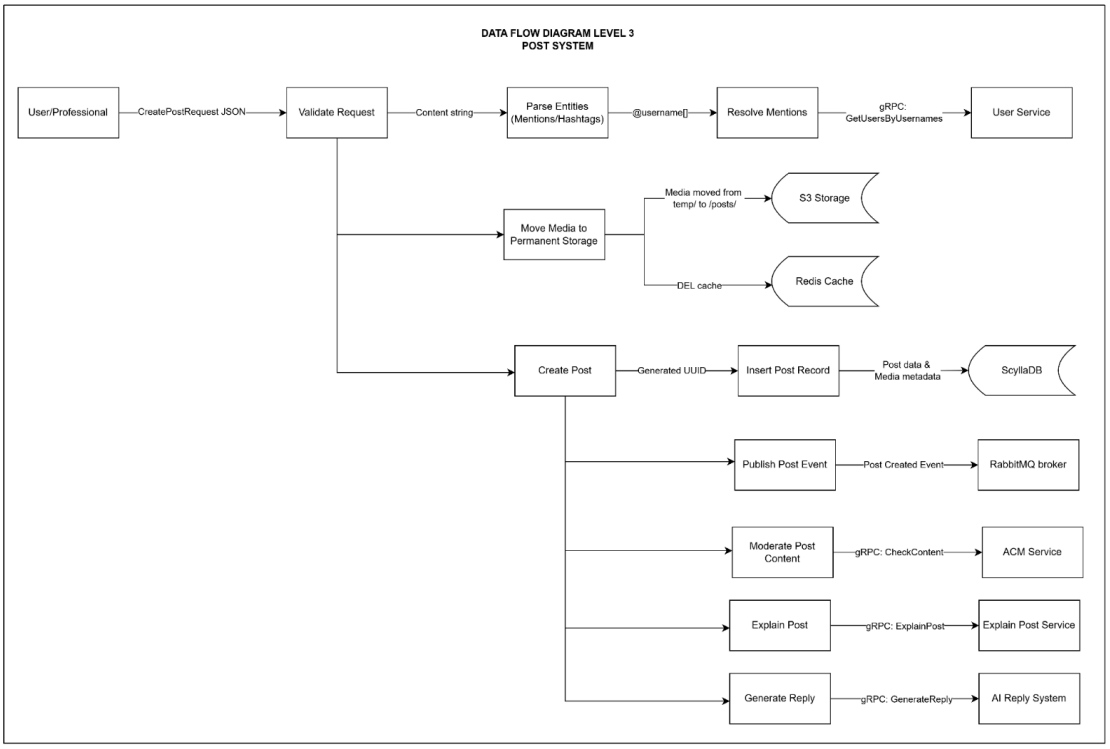


Figure 1.1.1.1 Data Flow Diagram Level 3 (Post System)

Explanation

The process starts when an authenticated user or a professional sends a CreatePostRequest JSON to the API endpoint. This JSON includes the text content of the post, a list of media file references (from a temporary S3 upload), and metadata like optional tags or settings. And then it will validate the request from the backend, such as required fields (e.g., content or media), authentication (e.g., via JWT), and user role (e.g., user or professional, or admin).

From the content string, the system extracts @username as a mention and #hashtag as a topic. These are later stored or used for linking/tags. The extracted @username values are passed to the User Service via a gRPC call GetUsersByUsernames. The User Service returns the corresponding UUIDs of the mentioned users.

Then, in Move Media to Permanent Storage, media files initially uploaded to temp/ in S3 are moved to "posts/{post_id}/…". Temporary keys are deleted from the Redis Cache to mark the upload complete.

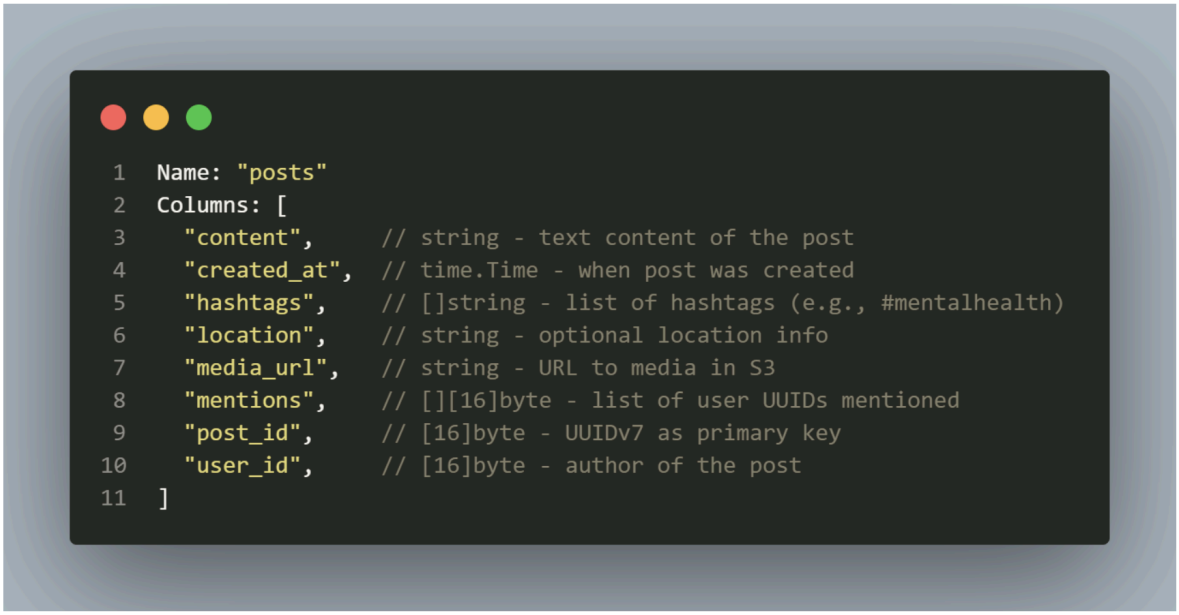
After that, when creating a post, a UUID is generated for the post, ensuring time-sortability and uniqueness. The system writes a full post object into ScyllaDB, including Post ID, Author ID, Content, Media metadata (S3 path, type, etc.),

Mentions and hashtags, Timestamp. A PostCreated event is published to RabbitMQ, using a fanout exchange.

In parallel (asynchronously), the post content is sent to the ACM Service (e.g., using DeepSeek) via gRPC "CheckContent(post_id, content, media)". The ACM Service returns moderation status. If it is safe, there is no action, but if it violates, it will mark the post as flagged or censored.

The post content is sent to the AI Reply System via gRPC to generate the fastest responses by AI. And it was sent to Explain Post via gRPC, also to explain the post content using a reasoning method. If successful, the responses from that AI will be displayed in the comment form.

1. ScyllaDB Schema



```
Name: "posts"
Columns: [
  "content",     // string - text content of the post
  "created_at",  // time.Time - when post was created
  "hashtags",    // []string - list of hashtags (e.g., #mentalhealth)
  "location",    // string - optional location info
  "media_url",   // string - URL to media in S3
  "mentions",    // [][16]byte - list of user UUIDs mentioned
  "post_id",     // [16]byte - UUIDv7 as primary key
  "user_id",     // [16]byte - author of the post
]
```

Figure 1.1.1.3 Pseudocode domain/domain.go

This file 'domain/domain.go' defines the ScyllaDB table schema for posts via gocqlx. The Go struct PostsStruct that maps to that schema, that used by repository/service layers to insert and query post data.

2. Database implementation

1. Post System

Technology Selection: ScyllaDB

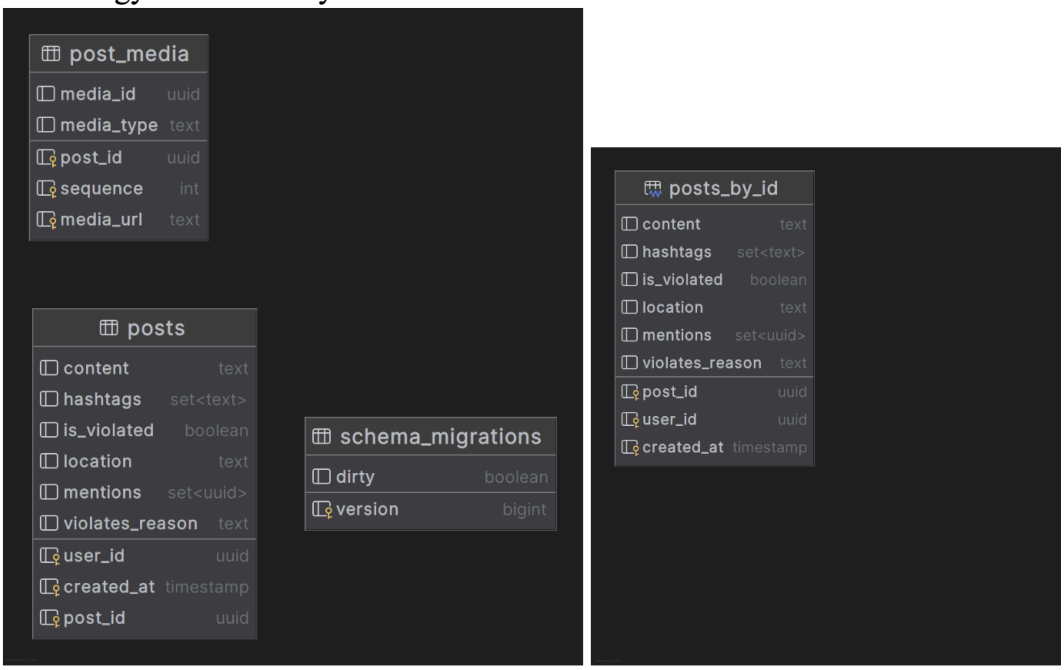


Figure 1.2.1.1 Database implementation Post System

We have selected ScyllaDB as our primary database technology. ScyllaDB, a high-performance NoSQL database compatible with Apache Cassandra, with advantages for the Post System use case:

- Exceptional Read Performance, whose architecture is optimized for low-latency reads, is crucial for delivering a responsive user experience in a social media-like platform where fetching posts is a frequent operation.
- Cassandra Compatibility: This allows us to leverage the mature Cassandra ecosystem, data modeling patterns, and driver support while benefiting from ScyllaDB's performance enhancements.
- Scalability, ScyllaDB's distributed nature ensures horizontal scalability, allowing us to handle growing data volumes and user loads effectively.

The advantage of our design is first the primary key (user_id, created_at, post_id) is fundamental to our performance strategy. user_id serves as the **partition key**. This means all posts by a single user are co-located on the same node (and its replicas). This design dramatically speeds up queries fetching a specific user's timeline or profile posts, a very common read pattern. created_at and post_id act as **clustering keys**. They define the sort order *within* each partition. Data is primarily sorted by created_at (allowing for easy chronological or reverse-chronological retrieval) and

secondarily by post_id (ensuring uniqueness). Second, we utilize appropriate data types, such as UUID for unique identifiers and set<text> or set<uuid> for efficient handling of multiple hashtags and mentions. Third, this structure directly supports efficient queries like "Get the latest N posts from user X" or "Get posts from user X within a specific time range."

3. User Interface implementation

The UI implementation of the sahari platform was based on user-centered design principles of clarity, accessibility, and emotional safety. These principles were converted to functional UI/UX decisions through the iterative design process that was outlined in the Form 3 document.

To maintain scalability and performance, the frontend is developed using the Next.js App Router, a modern full-stack React Framework with support for Server-Side Rendering (SSR) and Client-Side Rendering (CSR). The styling is managed using Tailwind CSS, with the ability to create utility-first, responsive designs, and UI components are developed with the help of ShadCN, a headless UI component library that is compatible with Radix and Tailwind.

The state management and data retrieval are handled with TanStack Query (React Query), which can support caching and background data synchronization. The application structure is based on Domain Driven Design (DDD) to ensure clean separation of concerns and scalability across various modules.

Before development began, UI prototypes were designed in Figma, where the design went through multiple feedback with stakeholders and also target users. Each page

and feature was developed to mirror the user's emotional process, especially Sahari's focus on emotional expression and mental well-being. Below is the breakdown of key UI components implemented.

1. Authentication System

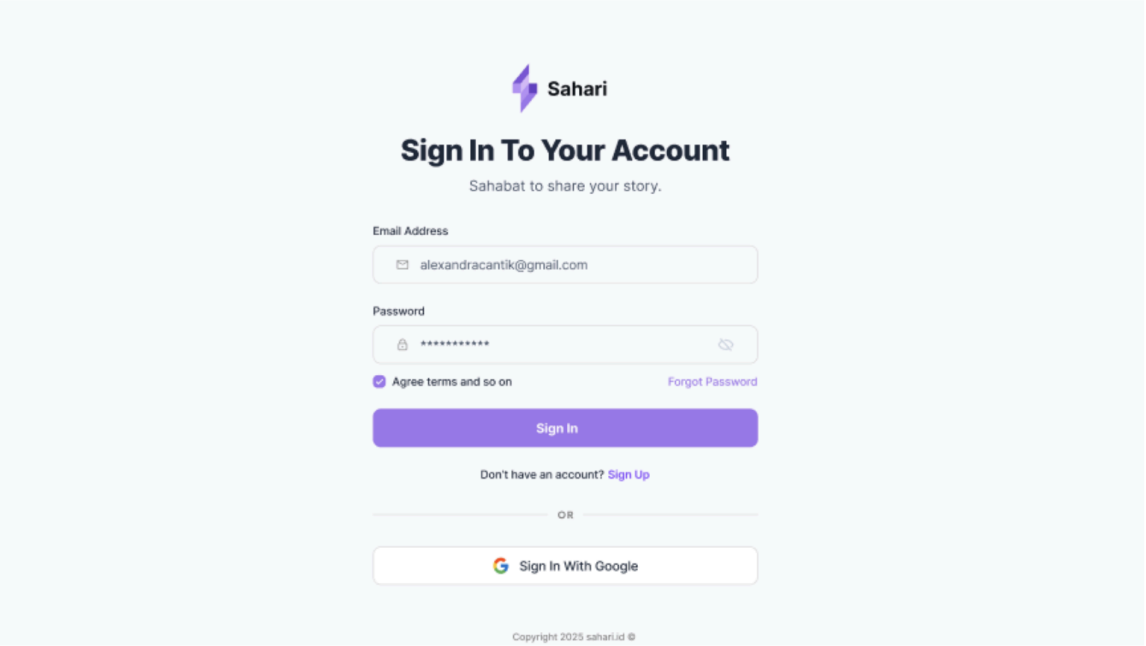


Figure 1.3.1 Sign In Page

The Comment system is powered by Dgraph to provide effective management of nested replies and threaded conversations. Comments link directly to post identifiers, and mention notifications are integrated with the User Service to resolve the username and make notifications via Redis or WebSocket events (for eventual real-time purposes).

The Feed Service integrates the output of the Follow, Post, and Like modules to create a user's personalized content stream. It utilizes RabbitMQ for fan-out architecture. When a user creates a post, it is pushed to their followers' feeds using asynchronous jobs. The Feed Service also has mixed ranking strategies (e.g., trending, most popular, or new-first) pulling metadata from the Like and Comment services.

Each interaction, from the generation of posts to the generation of AI responses, to discussion participation, is reflected in the frontend UI, which is developed with React using Next.js App Router. TanStack Query is used to integrate, so there is minimal user latency with regular data mutation and background syncing. Shared context

providers and dynamic components make UI updates occur seamlessly across modules.

All services are containerized with Docker and orchestrated with Docker Compose or Kubernetes in staging environments so that system consistency and deployment reliability can be preserved. This ensures that every module can independently be tested, scaled, and redeployed, but remain joined with each other through internal service discovery and environment-based configurations. The outcome is an integrated, scalable, and extensible architecture in which each module, from authentication through AI generation, supports a common mental health support service. This ensures the platform will remain responsive, reliable, and extensible as Sahari was created to become a nationwide digital program.

In conclusion, the integration between services in Sahari is achieved through secure authentication processes, asynchronous messaging, and service communication using REST and gRPC. Each module contributes to combining mental health support experience with independent scalability and maintainability. The architecture ensures that future modules, such as expanding AI support chatbot or analytics integration.

B. PRODUCT DISPLAY

1. SOFTWARE PRODUCT DISPLAY

Explain/description of each of the components. Show all the screenshots of the interfaces, including all the possible scenarios such as error messages, pop-ups, etc.

EXAMPLE

The Sahari platform is a web-based mental well-being ecosystem that provides user-generated expression, AI-supported guidance, professional consultation services, and social engagement. The software product is developed with a module-based, service-oriented architecture responsible for enabling vertical scalability as well as horizontal extendability.

The system is running as a full-stack solution, with a Next.js frontend, a series of Go-based microservices, and polyglot database connections (PostgreSQL, MongoDB, ScyllaDB, Redis, Dgraph). Below is the overview of how the product is structured, by module and user flow.

1. Authentication Page

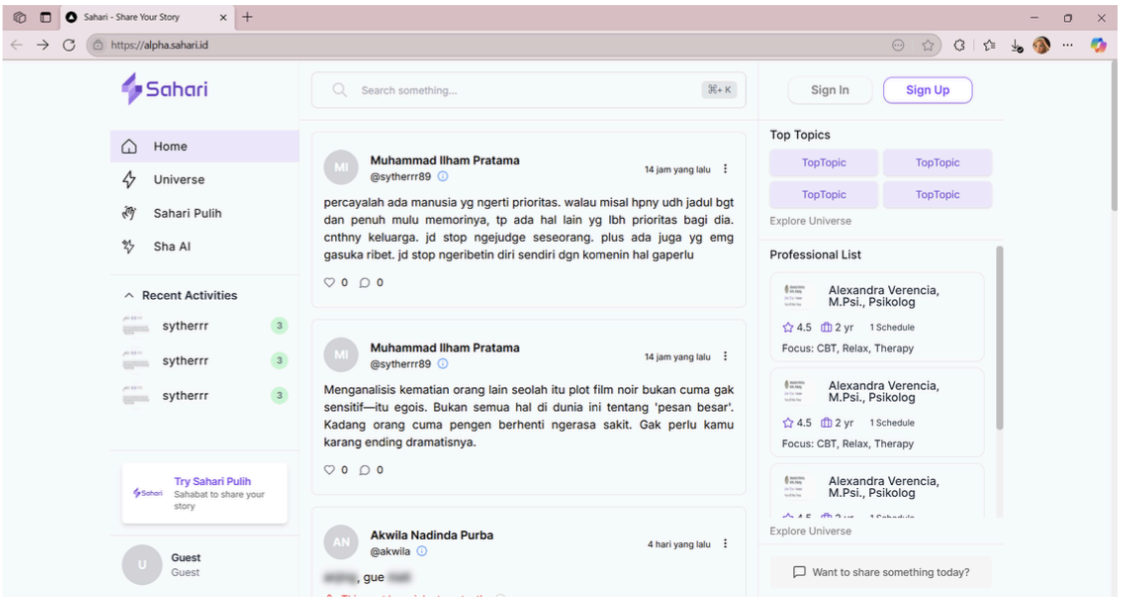


Figure 2.1.1.1 Home Page

The user will be shown this page first when accessing the web platform. Starting from the homepage of the Sahari Microblogging platform, will displayed the Feed System that contains the post content from trending users.

Users cannot interact with the content, nor can they like, comment, or post. The only possible actions are limited to viewing parts of the feed and another feature in Sahari.

This approach ensures that new users are exposed to the platform's core values (community expression and empathy) without requiring a login, while encouraging them to register.

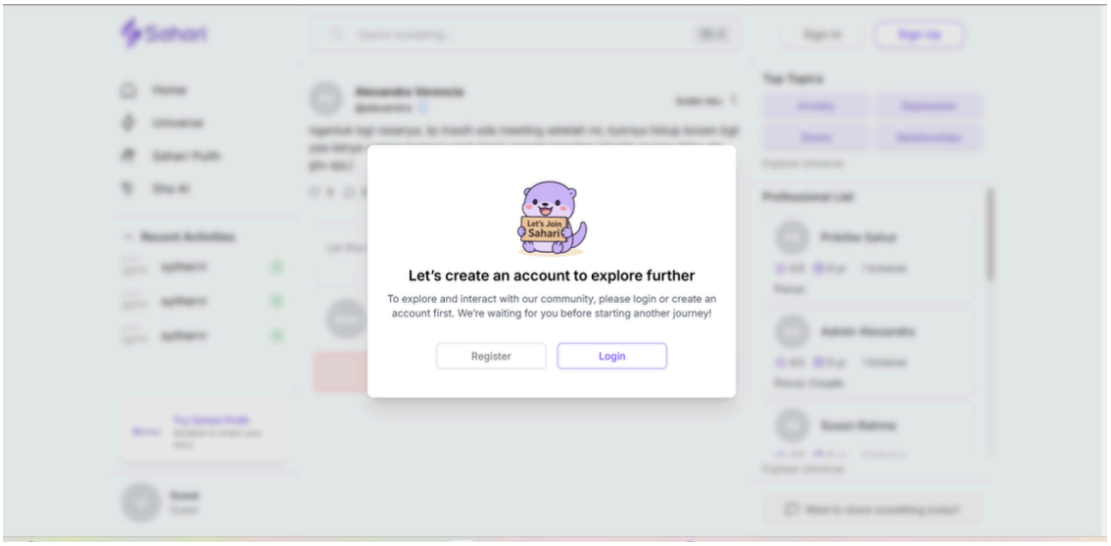


Figure 2.1.1.2 Login/Register Modal

The user can choose Sign in or Sign up (Login/Register) to access all of the features in Sahari.

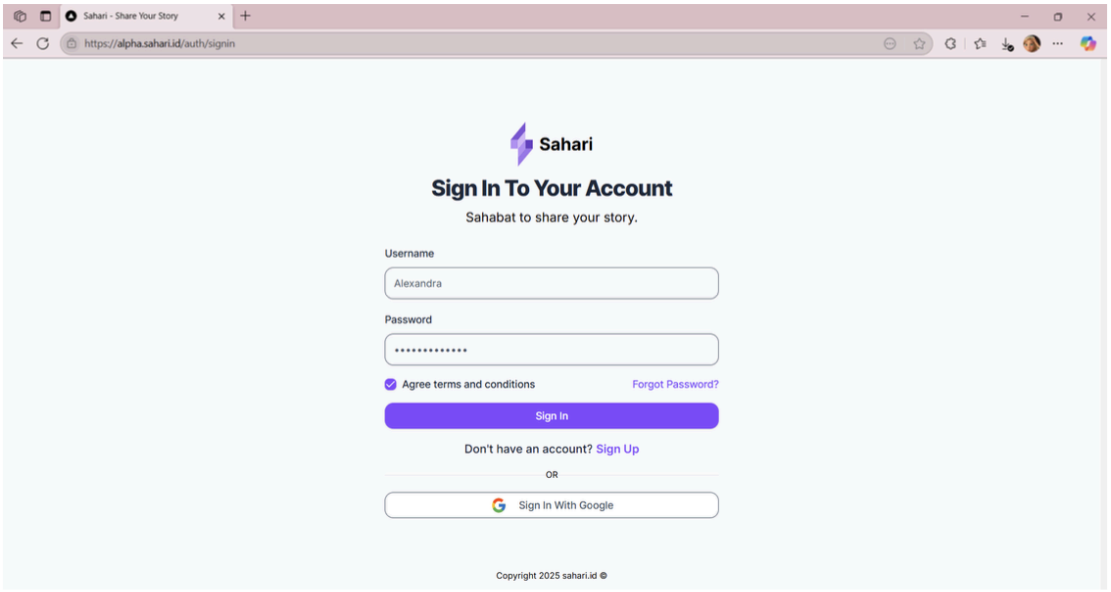


Figure 2.1.1.3 Sign In Page

In Sign In/Login, it will direct the user to the login form. The user must input a username and password, agree to the Terms and Conditions, and submit the login form.

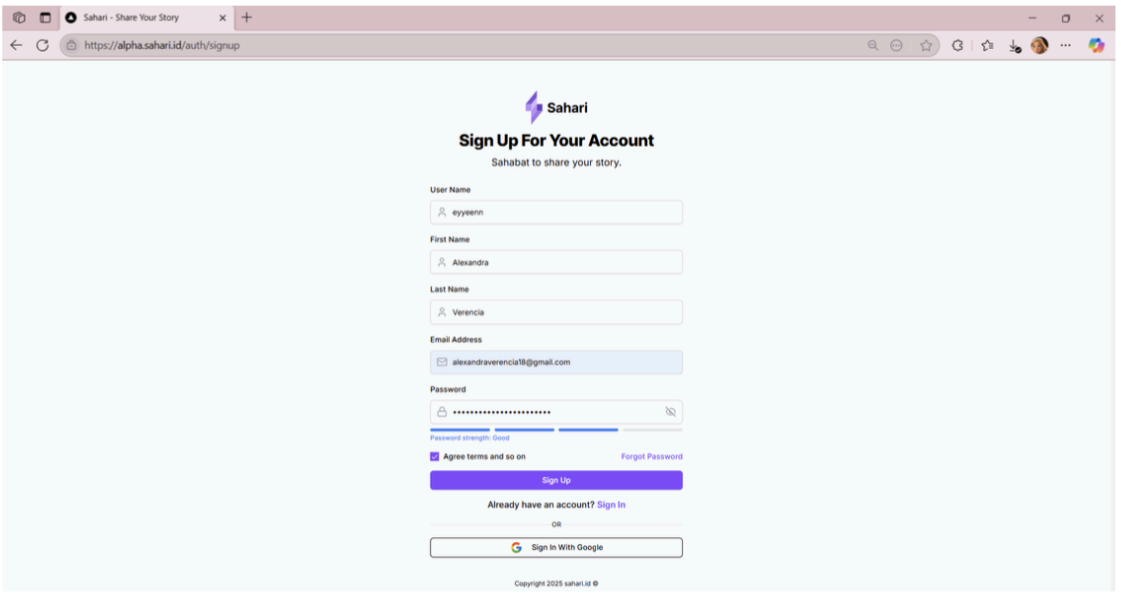


Figure 2.1.1.4 Sign Up Page

For new users, selecting "Register" or "Sign Up" leads to a Sign Up page requiring a username, email, password, and full name, and other optional fields.

2. Homepage (Sahari Space)

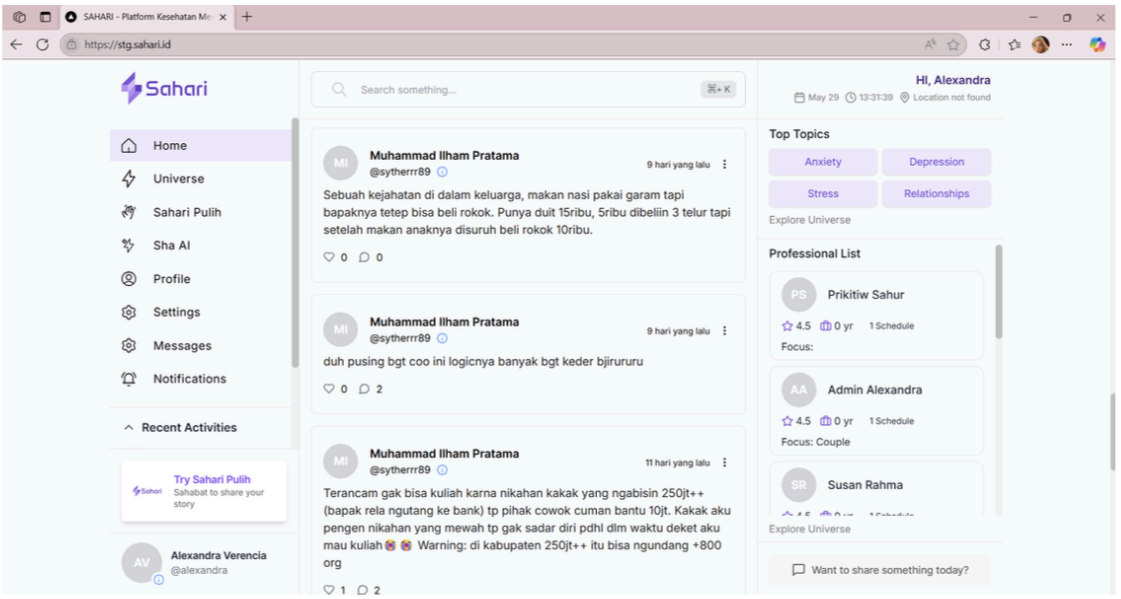


Figure 2.1.2.1 Feed/Home Page

The homepage (Sahari Space) presents a personalized feed of posts from followed users and recommended content.

2. HARDWARE PRODUCT DISPLAY

Explain/description of each of the components (if any). If you are creating an IoT-related project, you must put the circuit design in this section.

C. COMPONENT COST ANALYSIS

In this section, you must put all the necessary components to make this project work. This includes the server hosting approximation (if any)

No.	Item	Unit	Price per Unit	Total

EXAMPLE ONLY

No.	Item	Unit	Price per Unit	Total
1.	Server	/month	Rp. 145.000	Rp. 145.000
2.	Domain	/year	Rp. 210.000	Rp. 210.000
3.	DeepSeek API	/1M token output	Rp. 15.500	Rp. 15.500
4.	S3-compatible Storage (R2)	/ GB-month	Rp. 244	Rp. 244
Total				Rp. 370.744

Table 3.1 Component Cost Analysis

D. FUNCTIONAL TESTING

List all the tests done for all the functions in the project. You must put each feature in a separate table. Example: Log In process, Stock Monitoring process, Reporting process, each in a different table with all possible scenarios.

(Developer Perspective)

- **Prerequisites**
  - **Go 1.24+ (Official Docs)**
  - **Node 20+ (Official Docs)**
  - **Git (Official Docs)**
- **Clone the repository backend and frontend**
  git clone github.com/sahari/backend.git
  git clone github.com/sahari/frontend.git
- **Download the Go Module & Node Package Manager**
  go mod tidy
  npm install
- **Start All Backend Services**
  go run cmd/{name of service}/main.go
- **Copy .env.sample to .env & Edit values of NEXT_PUBLIC_API**
- **Start Frontend Services**
  npm run dev

No.	Scenario	Every Possible Input	Expected Output	Output Result

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	Log In with registered user	Email, Password, OTP	User logs in successfully	User logs in successfully
2	Log In with unregistered user	Email, Password, OTP	User fails to log in, show error	Error is shown and user cannot log in

1. SYSTEM BUILD DOCUMENTATION FROM THE SOURCE
   (Developer Perspective, this includes development program installation, how to run models, deploying docker containers, etc.)

   EXAMPLE

E. MANUAL GUIDE

2. END-USER SYSTEM INSTALLATION
   (Hardware, Software, Network installation from the user perspective).

(Hardware, Software, Network installation from the user perspective).

- **Prerequisites**
  - **Browser (Chrome, Safari, Edge)See their official docs**
  - **Internet with at least 3G Support**
- **Open your favorite browser and type the url box https://sahari.id**

3. USER GUIDE PER USER ROLE
   (How the user can use the system i.e. what to click, what to fill, etc.). Be reminded that this section contains the details of each user role (e.g. Officer, Admin) and what each role can do. Represent this in a table with details.

User Role	Page	Details
Admin	User Management	- Add User Admin can add user data and set another admin - Edit User Admin can edit user data and revoke privileges - Delete User - Show error in delete if user has existing posts

REFERENCES